capital

Plugin Development

Software Version 2015.1

February 2016

© 2016 **Mentor Graphics Corporation**
All rights reserved.

This document contains information that is proprietary to Mentor Graphics Corporation. The original recipient of this document may duplicate this document in whole or in part for internal business purposes only, provided that this entire notice appears in all copies. In duplicating any part of this document, the recipient agrees to make every reasonable effort to prevent the unauthorized use and distribution of the proprietary information.

This document is for information and instruction purposes. Mentor Graphics reserves the right to make changes in specifications and other information contained in this publication without prior notice, and the reader should, in all cases, consult Mentor Graphics to determine whether any changes have been made.

The terms and conditions governing the sale and licensing of Mentor Graphics products are set forth in written agreements between Mentor Graphics and its customers. No representation or other affirmation of fact contained in this publication shall be deemed to be a warranty or give rise to any liability of Mentor Graphics whatsoever.

MENTOR GRAPHICS MAKES NO WARRANTY OF ANY KIND WITH REGARD TO THIS MATERIAL INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE.

MENTOR GRAPHICS SHALL NOT BE LIABLE FOR ANY INCIDENTAL, INDIRECT, SPECIAL, OR CONSEQUENTIAL DAMAGES WHATSOEVER (INCLUDING BUT NOT LIMITED TO LOST PROFITS) ARISING OUT OF OR RELATED TO THIS PUBLICATION OR THE INFORMATION CONTAINED IN IT, EVEN IF MENTOR GRAPHICS CORPORATION HAS BEEN ADVISED OF THE POSSIBILITY OF SUCH DAMAGES.

RESTRICTED RIGHTS LEGEND 03/97

U.S. Government Restricted Rights. The SOFTWARE and documentation have been developed entirely at private expense and are commercial computer software provided with restricted rights. Use, duplication or disclosure by the U.S. Government or a U.S. Government subcontractor is subject to the restrictions set forth in the license agreement provided with the software pursuant to DFARS 227.7202-3(a) or as set forth in subparagraph (c)(1) and (2) of the Commercial Computer Software - Restricted Rights clause at FAR 52.227-19, as applicable.

Contractor/manufacturer is:
Mentor Graphics Corporation
8005 S.W. Boeckman Road, Wilsonville, Oregon 97070-7777.
Telephone: 503.685.7000
Toll-Free Telephone: 800.592.2210
Website: www.mentor.com
SupportNet: www.mentor.com/supportnet
Contact Your Technical Writer: www.mentor.com/supportnet/documentation/reply_form.cfm

TRADEMARKS: The trademarks, logos and service marks ("Marks") used herein are the property of Mentor Graphics Corporation or other third parties. No one is permitted to use these Marks without the prior written consent of Mentor Graphics or the respective third-party owner. The use herein of a third-party Mark is not an attempt to indicate Mentor Graphics as a source of a product, but is intended to indicate a product from, or associated with, a particular third party. A current list of Mentor Graphics' trademarks may be viewed at: www.mentor.com/terms_conditions/trademarks.cfm.

End-User License Agreement: You can print a copy of the End-User License Agreement from: www.mentor.com/terms_conditions/enduser.cfm.

Table Of Contents

Introduction **2**
Plugin Documentation 2
API Documentation 3
Templates and Examples 3
Plugin Folders **5**
Java Plugins **7**
Exercise 8
Guidance 11
Appendix A – Troubleshooting **13**

Introduction

This document describes the process for developing plugins for Capital.

This document is written in a tutorial style, meaning there will be many practical exercises for you to run. These exercises can be skipped on first reading; however, it is recommended that you do these exercises before developing your own plugins. To do the exercises, you need to have Capital installed. Please ensure that the installation you use when trying the exercises does not have any previously developed plugins installed. Apart from the application license you will also need a Capital Integration Server client license.

A plugin is a module that customizes Capital behavior in some way. All plugins implement specific interfaces, which $A = \pi r^2$ expressed in Java. To develop plugins for Capital, therefore, you need to be familiar with Java programming. If this is not the case, you need to get training in Java before continuing with this tutorial. It would also be useful if you were familiar with UML diagrams.

Plugin Documentation

This document focuses on the information that you need to know when developing any plugin. Other documents focus on what you need to know when developing a specific type of plugin. If you are interested in developing custom actions, for example, you need to read this document first, and then read the **CustomActionDevelopment.pdf**.

The following customizations are possible:

- **Custom Actions** – to develop your own menus that can optionally change properties and attributes on objects for Capital Logic, Capital Topology, Capital Integrator or Capital HarnessXC. If you are interested in this, please read **CustomActionDevelopment.pdf** after reading this document. Moreover, this is the best document to read next if you are interested in Capital plugins in general.

- **Custom Constraints** – to develop your own constraints for Capital Topology or Capital Integrator. If you are interested in this, please read **CustomConstraintDevelopment.pdf** after reading this document.

- **Custom DRCs** – to develop your own DRC checks for Capital Logic, Capital Topology, Capital Integrator or Capital HarnessXC. If you are interested in this, please read **CustomDRCDevelopment.pdf** after reading this document.

- **Custom Design Inspectors** – to develop custom views on the data held within objects for Capital Topology or Capital Integrator. If you are interested in this, please read **CustomDesignInspectorDevelopment.pdf** after reading this document.

- **Custom Authentication** – so you can provide alternate username/password authentication for Capital. If you are interested in this, please read **CustomAuthenticationDevelopment.pdf** after reading this document.

- **Custom Tables** – to develop custom tables in the design authoring applications. If you are interested in this, please read **CustomTableDevelopment.pdf** after reading this document.

- **Custom Reports** – to develop custom reports for Capital Enterprise Reporter. If you are interested in this, please read **CustomReportDevelopment.pdf** after reading this document.

- **Custom SBOM Operations** – to develop custom operations for Capital SBOM. If you are interested in this, please read **CustomSBOMOperationDevelopment.pdf** after reading this document.

API Documentation

The documentation for the Java interfaces (the API) can be found within the Capital installation folder in the following location:

```
doc/plugin/api/index.html
```

Moreover, the plugins you develop will be passed in "business" objects as parameters, from which the Capital information you require can be obtained. It is important to note that these business objects do not contain graphical information about objects, just the properties and attributes of the objects and their relationship with each other. In some situations, these properties and attributes can be modified. For more information about property/attribute modification, see *CustomActionDevelopment.pdf*.

Templates and Examples

Templates are provided so that beginners to plugin development have a starting point for creating new plugins.

Example plugins are also provided, which show how you might organize your plugins as they increase in number. You can also copy and modify the examples to implement your own plugins if you wish. However, you need to check the license and copyright information at the start of the files before you do this.

The templates and examples that are provided may be different in future releases of Capital. If you do use these examples, therefore, you need make a copy of them (we reserve the right to add, remove or modify these examples and templates).

Plugin Folders

All plugins should be placed within the Capital installation in the following folder:

```
plugins
```

When the Capital application starts, it will scan the plugins folder and load any plugins it discovers there.

Any plugins placed in that folder will be available to that installation only. If you require plugins be available to all users, you will need to ensure that all Capital installations have the same plugins folder.

It is recommended that when you develop these plugins, you do it on a separate installation so as not to affect other users whilst debugging. You must have write permissions on the Capital installation folder in order to develop plugins. Users of the plugins, however, will just need read access to those files.

Plugins can be placed in sub-folders under the plugins directory. This allows you to organize the plugins you develop in ways that best meet your needs. For example, if you need to develop plugins for different business units that your company supports, you can place those plugins in different sub-folders as follows:

```
plugins
plugins/general
plugins/BU1
plugins/BU2
```

Any of the above plugins folders can have an “extras” sub-folder that contains additional libraries that need to be loaded by the plugin. For example, all the plugins that are placed in this folder:

```
plugins/general
```

will be able to load all libraries placed in this folder:

```
plugins/general/extras
```

This ‘extras’ folder should never contain plugins, otherwise, those plugins will not be loaded.

Plugins can sometimes rely on the use of third party libraries such as Apache BATIK, for SVG manipulation. It is advised to keep these libraries in “extras” folder to ensure that capital framework doesn’t unnecessarily scan through these folders for plugins at startup.

Note: Capital ships with several third party libraries included in the lib folder (the full list can be seen in the release notes for the current release). If you want to use these libraries in the plugin please copy the corresponding jars to “extras” folder. Even if you don’t copy these it might work

but the next version of Capital may ship with a different version than the one your plugin needs and hence it is advised to copy the one as needed. The one caveat is for libraries that can only be loaded once in a single virtual machine (referred to here as restricted libraries). Some Apache libraries used by Capital require this type of restriction, if your plugin depends on any of these libraries **do not** copy them into the plugins folder.

Remember: Those people developing plugins for Capital will require write permissions on the Capital installation directory or they will be unable to develop their plugins. However, those using plugins do not need write permissions on the Capital installation directory.

Java Plugins

If performance is important, it is recommended that you develop your plugins in Java. This will be especially true if you are building custom constraints. You can skip this section if you do not wish to develop Java plugins.

Java plugins are jar files that contain one or more classes that implement a particular interface. To create a custom DRC for Capital Logic, for example, the class you develop would need to implement the `IXLogicDRCheck` interface.

Java plugins will need to be compiled into jar files and placed into a plugins folder. For example, if you have created several custom DRCs, you could compile them into a single jar file and place them into the plugins folder (or a sub-folder):

```
plugins/general/drc.jar
```

If this plugin accesses a database, it may require an additional jar file to be loaded that contains the driver for the database, this can be placed in the following folder:

```
plugins/general/extras/database_driver.jar
```

For performance reasons, it is strongly recommended that you place any jars that you depend on in an extras sub-folder (otherwise, these jars will be scanned for plugins).

To support Java development, you need to download the Java 1.8 SDK update 66 from:

http://java.sun.com/

It is also recommended that you use ANT and/or an IDE to build your plugins. ANT can be downloaded from:

http://ant.apache.org/

When building your own Java plugins, you will need to have `capitalapi.jar` in your classpath. This can be found within the capital installation folder in the following location:

```
lib/capitalapi.jar
```

You should not create plugins that exist in the following packages, otherwise those plugins may not load or may conflict with plugins provided by MCD:

- chs.
- library.
- uk.co.firstearth.

- com.mentor.

Exercise

In this exercise, we shall attempt to build the example Java plugins provided as part of the Capital installation.

1) You need to install the Java SDK (not just a JRE). Download the correct version (Java 1.8 SDK update 66) of the Java SDK from:

   http://java.sun.com/

   Follow the installation instructions for the Java SDK. Please ensure that your JAVA_HOME environment variable points to the Java SDK installation folder. Moreover, the bin folder in the Java SDK installation folder should be in your PATH environment variable when you start a UNIX/DOS console.

2) You need to install ANT. Download a copy of ANT, if you have not done so already, from

   http://ant.apache.org/

   Follow the installation instructions for ANT. Please ensure that your ANT_HOME environment variable points to the ANT installation directory. Moreover, the bin folder in the ANT installation folder should be in your PATH environment variable when you start a UNIX/DOS console.

3) The example Java plugins can be found within the Capital installation folder in the following location:

   ```
   doc/plugin/examples/Java
   ```

   From a UNIX console or DOS console, cd into the folder specified above.

   Ensure that your CAPITAL_HOME environment variable has been set in the console you are using and that it points to the CAPITAL installation directory (ANT_HOME and JAVA_HOME should also be set at this point).

4) To build the examples, use the following command:

   ```
   ant
   ```

   Eventually, you should see several messages as a result. The last one should be a “BUILD SUCCESSFUL” message.

This should have built the example jar and copied it to the following location in the Capital installation:

```
plugins/Examples/example.jar
```

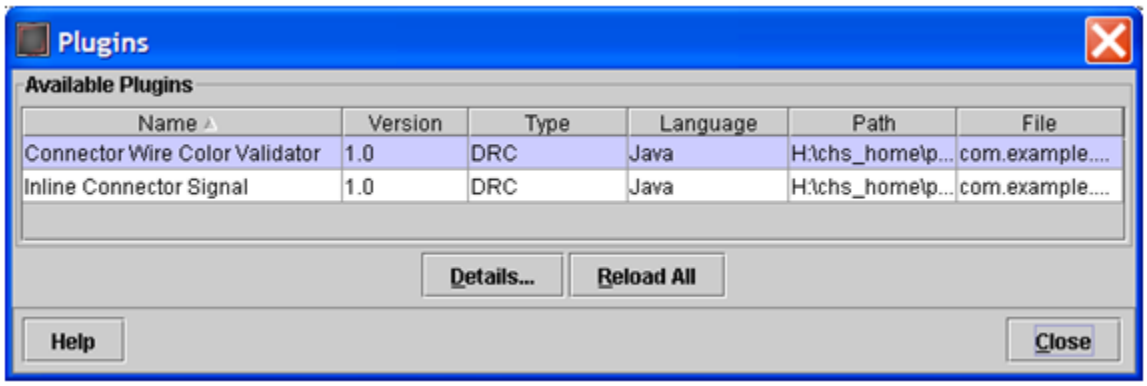
You should note that the build script you have just used automatically copies the successfully built plugin jar to the correct folder in the Capital installation. When you start building your own plugins (using ANT or an IDE), you will need to do something similar or manually copy the files you require.

5) Now you can check if the plugins can actually be loaded.

   Start Capital Logic, Capital Integrator, Capital HarnessXC or Capital Design.

   Once you have typed in the username/password, the application should be open.

   Click on the File -> Plugins… menu, this will display a dialog like the one shown below:



   This dialog displays information about the plugins that have been successfully loaded. This includes the name of the plugin, the version, etc.

   If you see an empty list, you may need to repeat the previous steps to ensure that the example plugins have been built. If you see a list of plugins, then you can continue. Other plugins may appear in this list, but you should see the two plugins shown above.

   Keep this dialog open for now.

6) Now we are in a position to create our first plugin. There is a template DRC plugin in the following location:

```
doc/plugin/examples/Java/TemplateDRC.java
```

Templates are provided so that beginners to plugin development have a starting point for creating new plugins. This particular template is the starting point for creating a custom DRC.

Copy this template into the following folder (please copy the file, do not move it):

```
doc/plugin/examples/Java/src
```

You should now have the following file:

```
doc/plugin/examples/Java/src/TemplateDRC.java
```

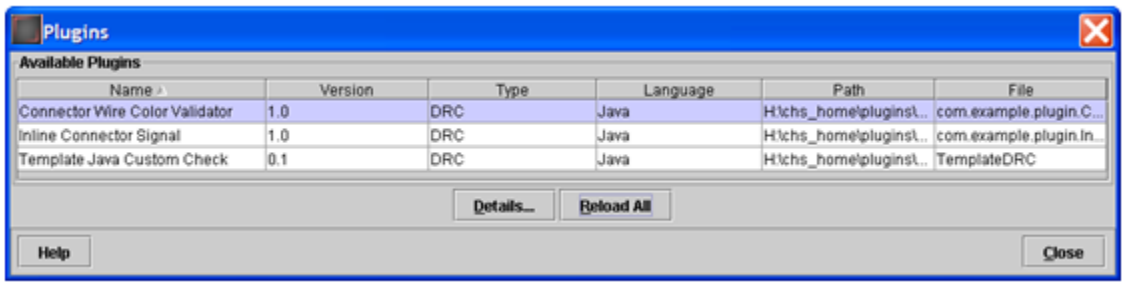
The folder `src` is the directory where all the example plugin source code has been placed. When you have completed all the exercises in this document, it would be a very good idea to look at the source code for the plugins that Capital has been distributed with. This will help you when you start developing your own plugins.

7) Build the examples using the following command:

```
ant
```

Again, you should get a “BUILD SUCCESSFUL” message.

8) With the Plugins dialog still open, press the [Reload All] button. You should notice that an extra plugin has been detected: “Template Java Custom Check”.



At this point, you can select this new plugin and press the [Details…] button. This will open a dialog that displays extra information about the selected plugin. In this dialog you will see a description field. The description displayed in the field is obtained from the `getDescription()` method on the Java plugin class.

9) Now that we have a new template in place, you can start modifying the template to meet your needs.

Edit the file:

```
doc/plugin/examples/Java/src/TemplateDRC.java
```

In this file, you will see a TemplateDRC class. You will see several methods in the class including: `getName()`, `getDescription()` and `getVersion()`. As you can see, the values returned by these methods are displayed in the plugins dialog.

10) As an experiment, change the name and version number of the plugin. Rebuild the plugins using ANT.

    After rebuilding the plugin, press the [Reload All] button.

    You should see the name and the version number displayed for your plugin change.

    NOTE: It is recommended that you update the version number whenever a change is made to a released plugin. You can use this information to ensure that users have the most up-to-date plugin you have developed.

11) Experiment with changing the name of the class and moving the class to another package.

    NOTE: you should place any plugin you create in a package (do not create classes in the default package).

12) After closing down the Capital application, to completely remove the example plugins from Capital, you can type:

```
ant clean
```

At this point, you should know the following:

- How to build Java plugins in general.
- How to use the provided template plugins as a starting point for developing new plugins.

Other documents will focus on building specific plugins like custom DRCs and custom constraints.

Guidance

Here are some recommendations that will help you when building Java Plugins:

- Using an IDE and/or ANT can speed the time it takes to develop your Java plugins.

- Ensure that `capitalapi.jar` is in the class path of your IDE or ANT. This contains all the interfaces that you need to implement. This can be found in the lib folder of the Capital installation.

- If you are familiar with ANT, the `doc/plugin/examples/Java/build.xml` file in the Capital installation directory can be used as a basis for your own build scripts.

- Do not place your plugin classes in the default package. Decide on the package structure you are going to use before you deploy your plugins and stick to it. It should be unique enough to avoid clashes with other companies that you might work with.

- Use the provided templates to get you started. They have comments in them that will help you.

- Always use Java for custom constraints to ensure performance.

- Capital must be able to create an instance of your java plugin when it needs to. To enable this, please ensure that the constructor for your plugin has no parameters and is public.

Appendix A – Troubleshooting

- Your plugin is not compiling?
    - Check that you have included the capitalapi.jar into your classpath.
- Is your plugin not loading?
    - Check the Output Window for errors.
    - Check that you have not copied the plugin into an extras folder.
    - Is the plugin you created in the Capital plugins folder? If not:
        - Check you have write permission on the Capital folder.
        - Copy your plugins into the Capital plugins folder.
    - Check that the plugin has a .jar file extension (lowercase). You cannot place class files in the plugin directory, they will not be loaded.
    - Check that the version of Java you are using for the plugins is not higher than the version of Java being used by Capital.
    - Check that you have a default constructor (the constructor with no parameters) for your plugin class and that it is public.
- Are you having problems removing your plugin?
    - You may need to close the Capital application that has loaded the plugin.
    - Check that you have permissions to remove the plugin.